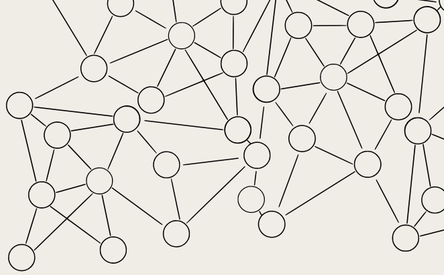
Adaptive Retrieval Orchestration for Retrieval-Augmented Generation using the CARO Framework in LLMs

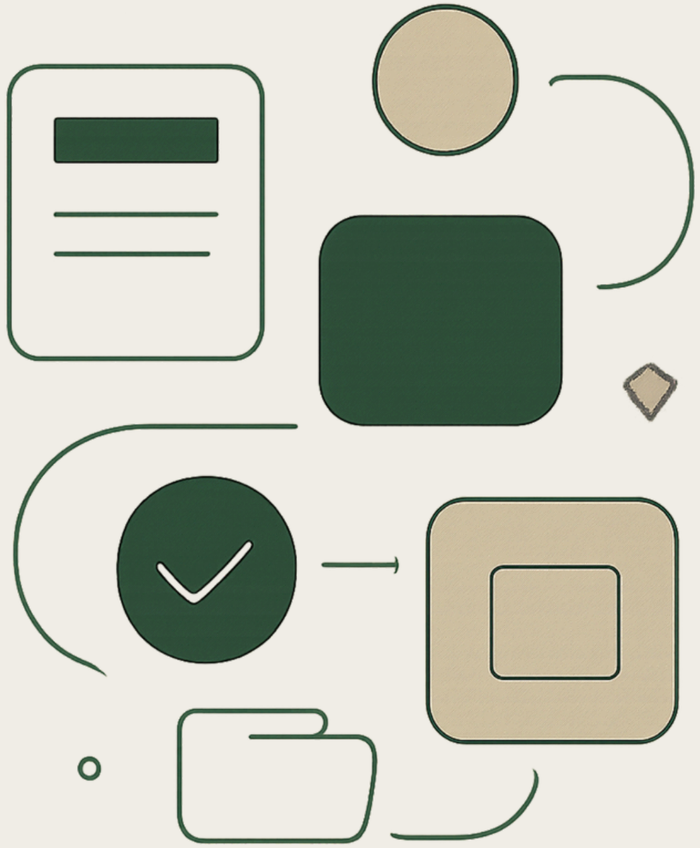
Presented By: Samprati
Register Number: 25MCA1059

Guide: Dr Rabindra Kumar Singh

Problem Statement

Current Retrieval-Augmented Generation (RAG) systems have improved the factuality and reasoning capabilities of Large Language Models. However, many existing RAG approaches still use retrieval mechanisms that are fixed or adapt only a specific part of the retrieval pipeline.





Direct answers

Some queries can be answered directly without retrieval.

Reuse validated knowledge

Some queries can be answered using previously validated information.

New retrieval needed

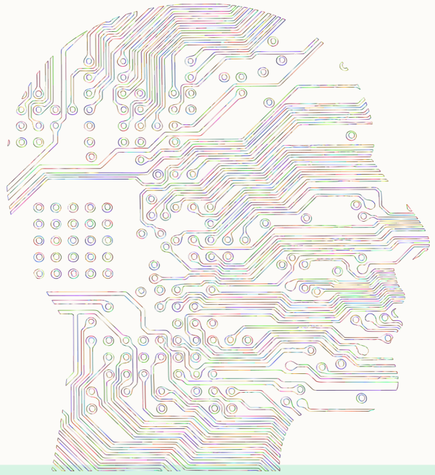
Some queries require new external document retrieval.

Iterative retrieval

Complex queries may require multiple retrieval attempts.

Noisy evidence

Retrieved documents may contain irrelevant or insufficient information.



These limitations can lead to:

- Unnecessary retrieval
- Repeated retrieval of the same information
- Irrelevant evidence
- Increased computational cost
- Reduced response reliability

Research Problem: There is a need for an adaptive orchestration mechanism that can intelligently decide when and how retrieval should be performed, reuse validated information, evaluate evidence sufficiency, and perform additional retrieval only when necessary.

Literature Review

Paper / Journal Name	Publisher / Publishing Year	Problem Statement	Methodology	Conclusion	Limitation
Retrieval-Augmented Generation for AI-Generated Content: A Survey	Springer Nature — *Data Science and Engineering* *2026*	RAG research has many architectures and techniques, making the field difficult to organize and compare.	Reviews RAG architectures, retrieval methods, applications, benchmarks, challenges, and future directions.	Provides a comprehensive view of RAG and highlights adaptive, iterative, recursive, and agentic approaches.	Survey-based work; it does not propose or experimentally validate one unified adaptive framework.
Context-Aware Retrieval Augmented Generation Using Similarity Validation to Handle Context Inconsistencies in LLMs	IEEE — *IEEE Access* *2025*	Conventional RAG may retrieve content that is similar but not sufficiently relevant to the actual context.	Uses context retrieval and similarity validation to check whether retrieved information supports the generated answer.	Similarity validation improves contextual relevance and helps identify poorly supported answers.	Additional validation and larger contexts increase computational cost and response time.
An Empirical Evaluation of Retrieval, Reranking, and Similarity for a Q&A-Based RAG System	IEEE — IEEE Access 2025	The impact of retrievers, rerankers, similarity metrics, and indexing strategies on RAG performance is unclear.	Experimentally compares different retrieval, reranking, similarity, indexing, and generation configurations.	Retriever and similarity choices strongly influence performance, while reranking can improve answer quality.	Evaluation uses limited datasets and fixed retrieval configurations, reducing generalization.

Literature Review

Paper / Journal Name	Publisher/ Publishing year	Problem Statement	Methodology	Conclusion	Limitation
Bias Amplification in RAG: Poisoning Knowledge Retrieval to Steer LLMs	IEEE — *IEEE Transactions on Dependable and Secure Computing 2026*	Poisoned knowledge retrieval can manipulate RAG outputs and amplify existing LLM biases.	Generates adversarial documents and manipulates retrieval and ranking to measure effects on model bias.	Shows that retrieval poisoning can amplify bias and makes retrieval an important security concern.	Focuses mainly on security and bias rather than adaptive RAG orchestration.
Leveraging HyDE and RAG in Gemma LLM Framework for Enlightened Deep Learning Chatbots	Springer Nature — *Human-Centric Intelligent Systems 2025*	Traditional RAG can suffer from semantic mismatch, hallucination, and weak domain-specific responses.	Uses HyDE for hypothetical-document generation, semantic retrieval, and LoRA fine-tuning of Gemma 7B.	HyDE combined with RAG improves domain-specific retrieval and response quality.	Evaluated mainly in an educational/deep-learning domain, limiting broader generalization.
iS3Tunnel: Highway-Tunnel Domain RAG-Enhanced LLM and Its Hallucination-Quantified Agent Design	Elsevier / KeAi — *Underground Space 2026*	General LLMs lack specialized engineering knowledge and may hallucinate on highway-tunnel questions.	Combines vector retrieval, knowledge-graph retrieval, an LLM agent, and multiple reasoning attempts.	Improves domain-specific question answering while providing hallucination and uncertainty assessment.	Highly domain-specific and requires additional knowledge-graph and reasoning resources.

Literature Review

Paper / Journal Name	Publisher/ Publishing Year	Problem Statement	Methodology	Conclusion	Limitation
A Hallucination-Resistant RAG System for Software Update Queries	Elsevier — *Procedia Computer Science* *2026*	RAG can hallucinate when software-update evidence is incomplete, outdated, contradictory, or ambiguous.	Checks query intent, source validity, evidence relevance, and freshness before generating or abstaining.	Evidence gating reduces hallucination by generating answers only when sufficient evidence exists.	Small evaluation and narrow focus on software-update queries.
AdaKG-RAG: Adaptive KG-Guided RAG with Hypothesis-and-Verification Retrieval for Multi-Hop QA	Springer Nature — *Journal of King Saud University Computer and Information Sciences* *2026*	Multi-hop RAG may retrieve relevant but incomplete evidence without checking whether evidence is sufficient.	Uses KG expansion, hypothesis-and-verification retrieval, sufficiency checking, re-retrieval, and fact reranking.	Improves answer quality and evidence coverage through adaptive additional retrieval.	Iterative retrieval increases latency and depends on knowledge-graph quality.
Self-RAG: Learning to Retrieve, Generate, and Critique Through Self-Reflection	ICLR 2024	Fixed RAG retrieval can be unnecessary and retrieved passages may not adequately support the generated answer.	Uses reflection tokens to decide retrieval, evaluate evidence, and critique generated responses.	Improves retrieval decisions, factuality, citation accuracy, and generation quality.	Requires specialized training and additional retrieval/critique computation.

Literature Review

Paper / Journal Name	Publisher/ Publishing Date	Problem Statement	Methodology	Conclusion	Limitation
AT-RAG: An Adaptive RAG Model Enhancing Query Efficiency with Topic Filtering and Iterative Reasoning	arXiv 2024	Complex multi-hop queries may retrieve many irrelevant documents, increasing cost and reducing efficiency.	Uses query topic classification, topic-guided retrieval, and iterative retrieval with reasoning.	Topic filtering improves efficiency while iterative retrieval improves multi-hop QA.	Mainly evaluated on multi-hop QA and iterative retrieval adds computational cost.
Agentic Self-RAG: Multi-Agent Reasoning for Self-Correcting RAG	IEEE — ICIT 2026	Standard RAG and single-model Self-RAG may struggle with evidence verification and self-correction.	Uses multiple agents for query analysis, retrieval criticism, evidence verification, confidence checking, and refinement.	Multi-agent orchestration improves evidence verification, reliability, and hallucination control.	Multiple agents increase LLM calls, token usage, and latency.
Dynamic Memory Retrieval in RAG Models: Enhancing Long-Context Reasoning	IEEE — ICAICE 2025	Static retrieval does not effectively reuse retrieval history or adapt to evolving long-context interactions.	Uses short-, mid-, and long-term memory with a memory controller based on contextual signals.	Dynamic memory enables more adaptive retrieval and supports long-context reasoning.	Multi-level memory increases system complexity and computational requirements.

Research Objectives

Primary Objective

Build an adaptive retrieval orchestration framework for Retrieval-Augmented Generation systems.

Specific Objectives:

1. Route

Develop an Adaptive Query Router (AQR) to determine the appropriate execution path for each query.

2. Classify

Classify queries according to their characteristics and complexity.

3. Retrieve

Select an appropriate retrieval strategy, including Context-Oriented Retrieval and Adaptive Retrieval.

4. Filter

Evaluate retrieved documents for relevance and remove irrelevant evidence.

5. Validate

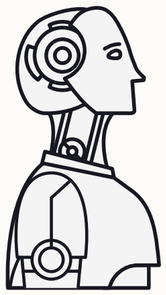
Maintain temporary memory containing only validated and relevant retrieved information.

6. Check Sufficiency

Perform evidence sufficiency checking and trigger additional retrieval when the available evidence is insufficient.

7. Evaluate

Evaluate the generated response for relevance, faithfulness, factual consistency, and confidence.



PROPOSED SOLUTION

Proposed Solution – CARO Framework

Framework: CARO – Context-Aware Adaptive Retrieval Orchestrator
CARO introduces an orchestration layer that dynamically manages the retrieval lifecycle instead of following a fixed RAG pipeline.

- Query Analysis
- Adaptive Router
- Query Classification
- Retrieval Strategy



CARO dynamically routes each query, validates retrieved evidence, and loops back when more information is needed before generating the final answer.

Query Analysis

Determine whether retrieval is required.

AQR Routing

Send the query to direct generation, temporary memory, or external retrieval.

Classification & Strategy

Identify query complexity and select the best retrieval strategy.

Retrieve & Validate

Fetch documents, filter relevance, and store only validated evidence.

Check, Loop, and Answer

Assess sufficiency, re-retrieve if needed, then generate and evaluate the answer.

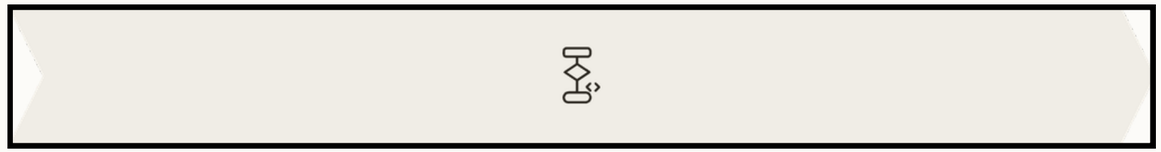
Proposed System Architecture

The architecture centers on CARO as an orchestration layer that routes queries, manages retrieval, validates evidence, and generates responses adaptively.



Query Analysis

Detect intent and determine whether retrieval is needed.



Adaptive Query Router

Send requests to direct generation, memory, or external retrieval.



Retrieval & Filtering

Classify, retrieve, and filter relevant documents.

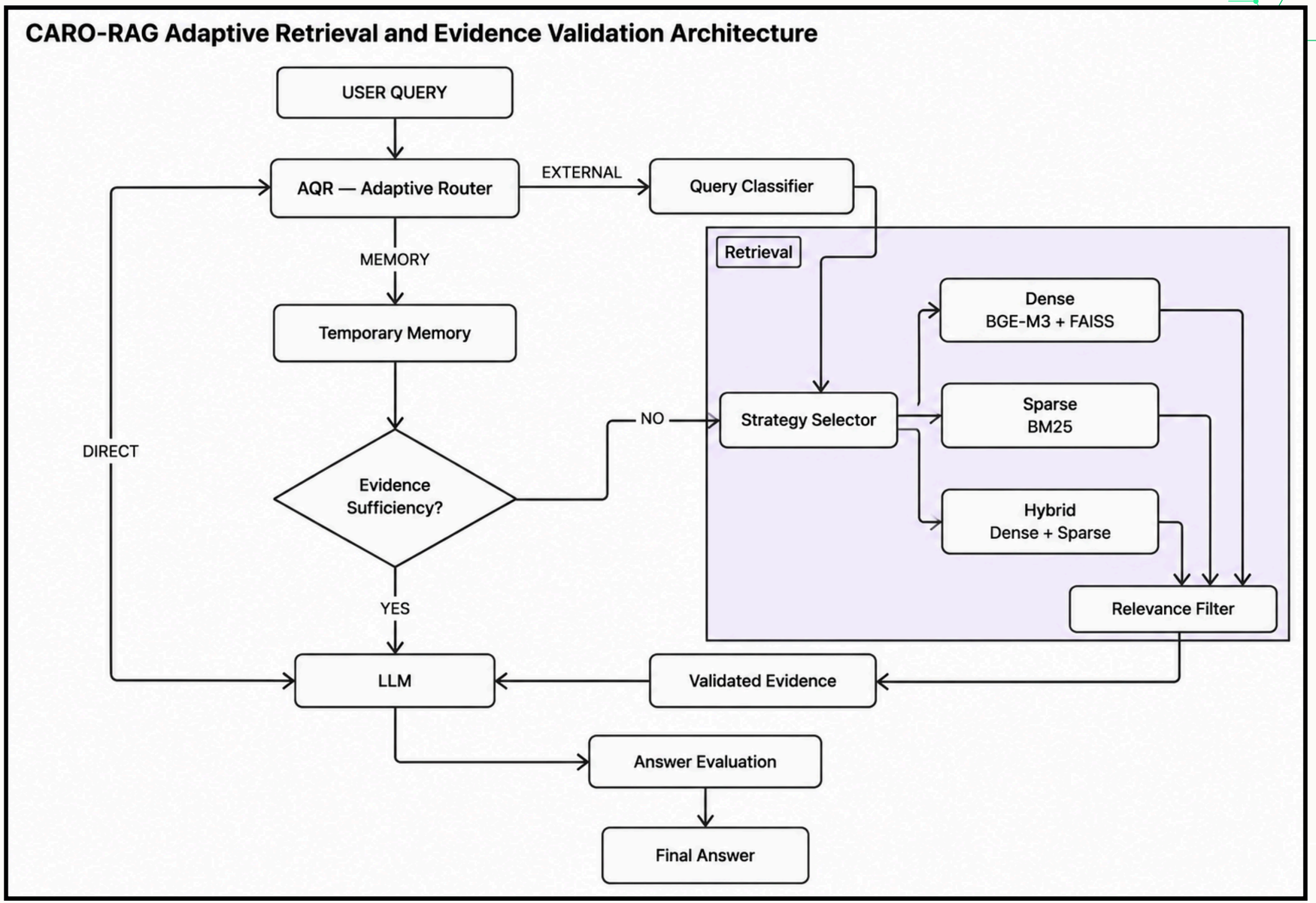


Evidence Check

Validate sufficiency and trigger re-retrieval if needed.

Validated evidence is stored in temporary memory and passed to answer generation, creating an adaptive loop that improves reliability and reduces unnecessary retrieval.

Proposed System Architecture



Dataset and Evaluation Plan

The CARO framework will initially be evaluated using two datasets.

1. HotPotQA

Purpose:

- Multi-hop question answering
- Evidence retrieval
- Complex reasoning
- Adaptive retrieval evaluation
- Evidence sufficiency and iterative re-retrieval

Why it is relevant:

HotpotQA contains questions that require information from multiple pieces of evidence, making it useful for evaluating whether CARO can retrieve and combine sufficient evidence for complex queries.

2. QRECC

Purpose:

- Conversational question answering
- Context-dependent queries
- Follow-up questions
- Temporary memory evaluation
- Context-aware retrieval

Why it is relevant:

QReCC contains conversational question-answering scenarios where later questions depend on previous conversation context. This makes it useful for evaluating CARO's ability to reuse previously validated information through temporary memory.

Modules of the CARO Framework

Module 1 – Document Ingestion and Indexing

- Document Loading
- Text Preprocessing
- BGE-M3 Embeddings
- FAISS Indexing
- Similarity Search

Module 2 Query Processing

- Query Preprocessing
- Query Analysis
- Semantic Representation
- Information Extraction

Module 3 Hybrid Retrieval

- Dense Retrieval
- Lexical Retrieval
- Relevance Ranking
- Similarity Scoring

Module 4: Adaptive Retrieval Orchestration

- Adaptive Routing
- Score Analysis
- Conditional Execution
- Retrieval Coordination

Module 5: Evidence Assessment

- Evidence Evaluation
- Sufficiency Classification
- Threshold-Based Decisions
- Partial Evidence Detection

Module 6: Temporary Episodic Memory

- Context Storage
- Context Retrieval
- Temporary Memory
- Unit-Tested Module

Module 7: Answer Generation

- Gemma 3 4B
- Evidence Conditioning
- Prompt-Based Generation
- Context-Grounded Answers

Module 8: Answer Validation and Abstention

- Response Validation
- Selective Abstention
- Evidence-Based Decisions
- Qualified Responses

Implementation

The core CARO RAG pipeline has been implemented and executed, covering document indexing, query processing, retrieval, adaptive orchestration, evidence assessment, answer generation, and abstention.

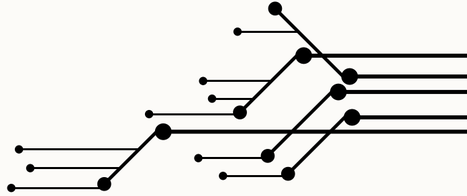
Technology Stack

- Language: Python
- Embedding Model: BGE-M3
- Generation Model: Gemma 3 4B (gemma3:4b)
- Vector Search: FAISS
- Benchmark Dataset: HotpotQA
- Evaluation Setup: Comparative testing against a baseline RAG system

Implementation Achievements

- Built a retrieval index containing 250,000 documents for the main comparative benchmark.
- Executed the baseline and CARO systems on 500 benchmark questions.
- Integrated evidence sufficiency assessment and selective abstention.
- Recorded answer outputs, retrieval results, latency, and evaluation metrics.
- Corrected inconsistencies between implementation artifacts, experimental results, and thesis documentation.

Implementation (Cont.)



Implementation Status:

- Core Pipeline: Completed
- All Modules: Implemented
- Benchmark Testing: Completed
- Episodic Memory: Completed
- Evaluation Metrics: Completed
- Results and Discussion: Completed
- LLM-Judge Evaluation: Completed

Key Implementation Outcome:

CARO RAG implements evidence-aware answering and selective abstention, enabling the system to answer, qualify, or withhold responses based on evidence sufficiency.

Evaluation Metrics used

The proposed CARO RAG system was compared against a baseline RAG system using the following metrics:

- **Exact Match (EM):** Measures the percentage of predictions that exactly match the reference answer.
- **Token-level F1 Score:** Measures the overlap between predicted and reference answer tokens.
- **Recall@5:** Measures how often relevant gold documents appear among the top five retrieved results.
- **Precision@5:** Measures the proportion of the top five retrieved documents that are relevant.
- **Mean Reciprocal Rank (MRR):** Evaluates the ranking position of the first relevant retrieved document.
- **Answer Rate and Abstention Rate:** Measure the proportion of questions answered and questions for which the system abstains.
- **Mean Latency:** Measures the average time required to process a question.

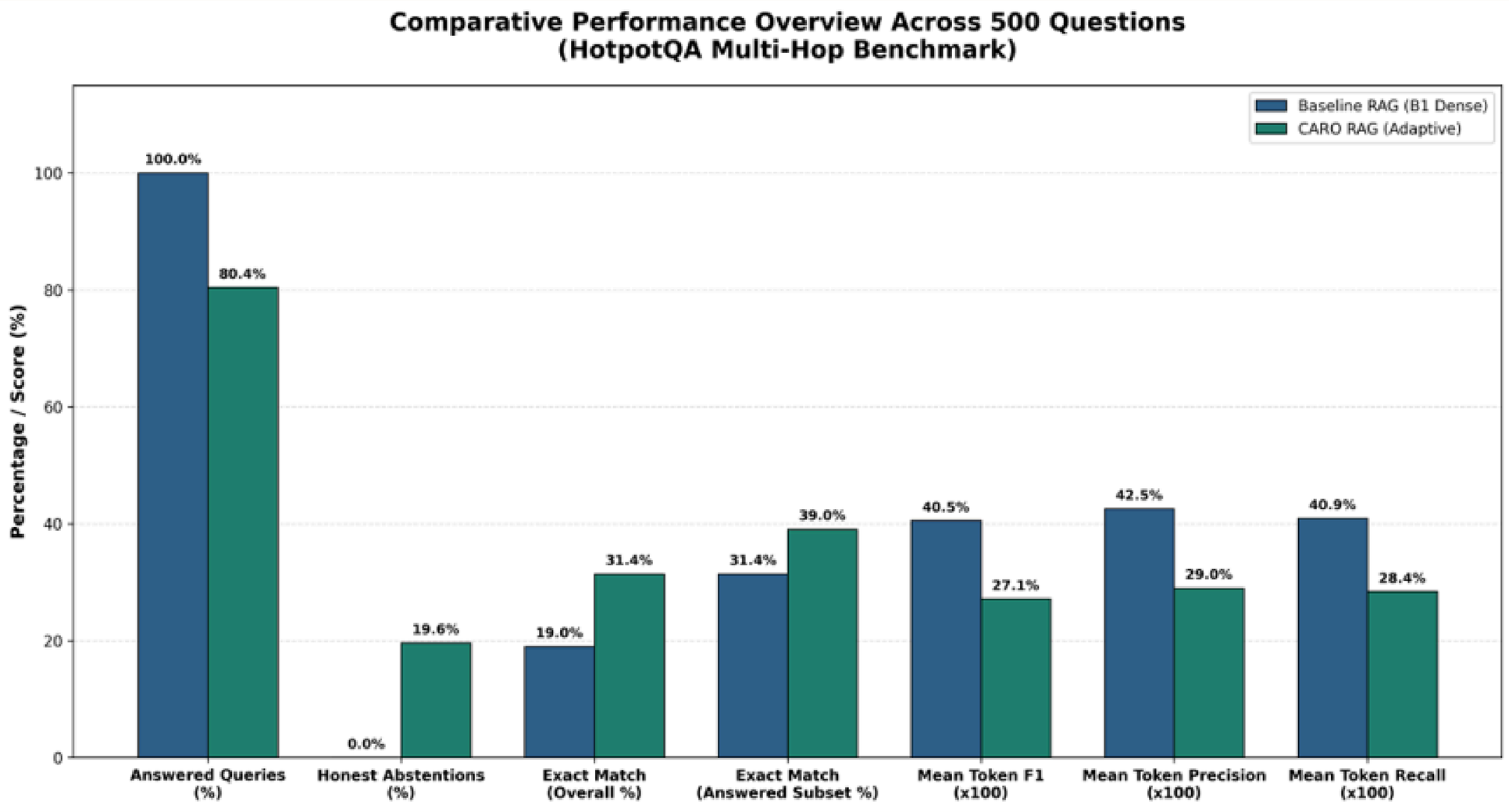


Results & Discussions

Baseline vs. CARO RAG

Metric	Baseline RAG	CARO RAG
Questions evaluated	500	500
Questions answered	500 (100%)	402 (80.4%)
Questions abstained	0 (0%)	98 (19.6%)
Exact Match	19.00%	31.40%
Token F1	0.2713	0.4052
Mean latency	25.02 s	64.48 s
Recall@5	0.284	0.4088
Precision@5	0.2898	0.4254
MRR	0.8537	0.904

Results & Discussions



Thank You

